Weight Assignment Logic

A] Introduction (Memristor Physics):

Memristor arrays form the foundation of next-generation neuromorphic hardware, enabling analogue computation through resistive states that retain memory of prior electrical activity [1], [2]. Each memristor functions as a tuneable weight element, directly influencing current flow and signal modulation across the network. The precise assignment of weights within such arrays is essential to ensure distinct conductance values and to maintain computational stability [3].

B] Objective:

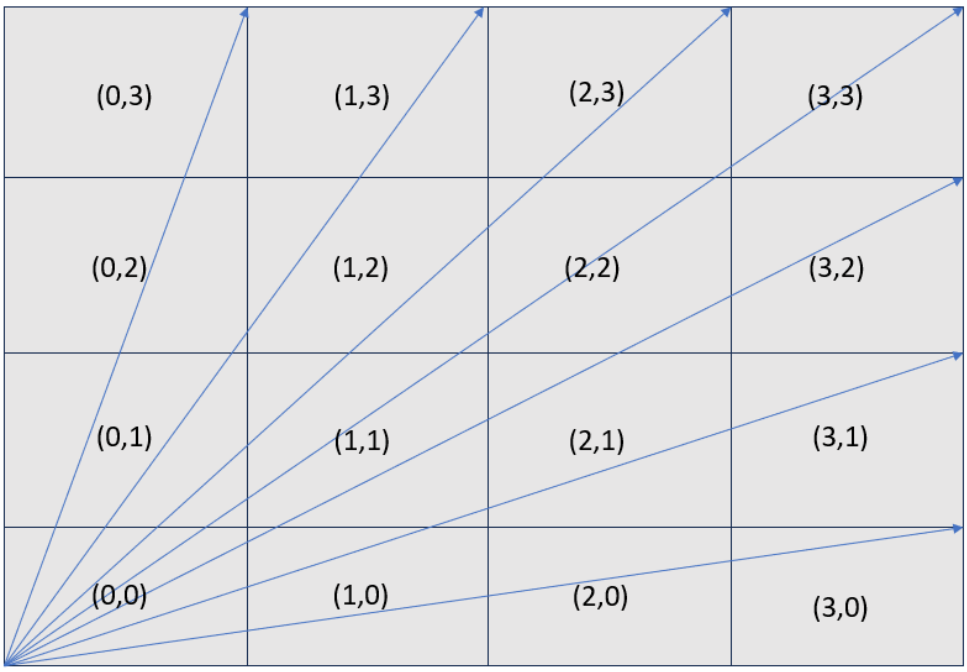
Assigning weights to $(n \times n)$ memristor arrays with unique values and unique sum is a challenging mathematics problem. To address the problem, we need a multiparameter diverging function which does not grow exponentially large in each given domain of values.

C] Methodology/Conceptualization of the initial test model:

Taking inspiration from complex coordinates, the function can be an exponential function so that if calculations were to be made in a logarithmic scale, evaluations will be simpler. Therefore, the function must be:

$$weights = e^{f(param)}$$

Since the function does not have to be exponentially large, we need to assign a "limiting factor" in the denominator of the function. However, to come up with a function it is important we visualize the grids, say a (4×4) matrix. One common factor that can help simplifying the problem is by drawing vector lines to see the behavior of a multiparameter function. The mapping can be visualized as follows:



$$\begin{pmatrix} (x_0, y_m) & \cdots & (x_m, y_m) \\ \vdots & \ddots & \vdots \\ (x_0, y_0) & \cdots & (x_m, y_0) \end{pmatrix}$$

Keeping these factors into consideration, we want a function that satisfy a few more check boxes (apart from the ones mentioned in the introduction) which are:

1. Define the parameters.
2. Defining the limiting factor.
3. Differentiate between the different vector lines.

The coordinates in the grid space can be described as (x_i, y_j). Here i and j are different coordinates in the grid. Therefore, a limiting equation can be derived from this concept. The proposed denominator is:

$$denominator = \sqrt{(x_{lm} - x_0)^2 + (y_{lm} - y_0)^2}$$

Here, x_{lm} and y_{lm}, is the coordinate of the last grid in a selected arrow and x_0 and y_0 are the coordinates at the origin.
This helps us giving flexibility in dealing with the numerator. Basis this, we can define the arbitrary numerator as:

$$numerator = (x_{li} + x_0 + 1) \times (y_{li} + y_0 + 1)$$

Therefore, the equation of weights can be written as,

$$weights = e^{\frac{(x_{li} + x_0 + 1) \times (y_{li} + y_0 + 1)}{\sqrt{(x_{lm} - x_0)^2 + (y_{lm} - y_0)^2}}}$$

The issue with this expression is that it does not differentiate different lines, therefore we need a factor of theta which handles it. The expression so far makes the points on the line unique, therefore adding theta to the expression will solve the problem of not having unique values in different lines. Therefore, the final expression can be written as:

$$weights = W_{(i,j)} = \theta + e^{\frac{(x_{li} + x_0 + 1) \times (y_{li} + y_0 + 1)}{\sqrt{(x_{lm} - x_0)^2 + (y_{lm} - y_0)^2}}}$$

θ ranges from 0 to $\frac{\pi}{2}$

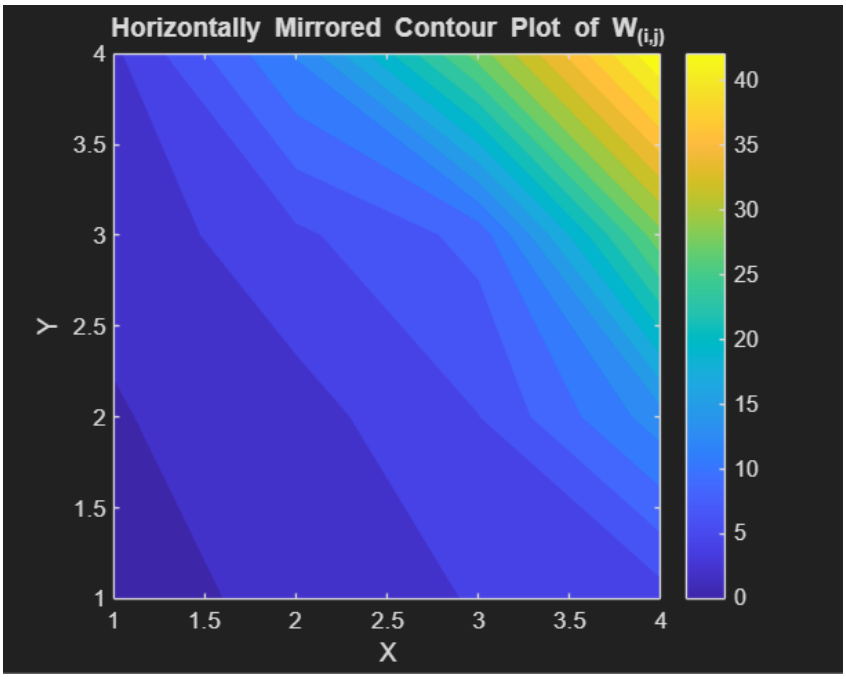
However, if we blindly apply the formula then the equation will result in different values at the origin. Therefore, we fix the origin with a weight of 0.

$$W_{(i,j)} = \begin{cases} 0 & at\ i = j = 0 \\ \theta + e^{\frac{(x_{li} + x_0 + 1) \times (y_{li} + y_0 + 1)}{\sqrt{(x_{lm} - x_0)^2 + (y_{lm} - y_0)^2}}} & elsewhere \end{cases}$$

For a (4×4) matrix, we can evaluate for each grid as:

5.3651	13.8	28.871	44.2194
4.2897	6.2637	9.1278	28.4761
3.5192	3.3529	5.8689	12.873
0	1.9477	2.7183	3.7936

On a contour plot the values can be mapped as:



D] Challenges and future works:

The challenges in this formula are that, certain angles, especially near $\frac{\pi}{4}$. Therefore, the function needs to be refined to keep the function from blowing up for certain lines. Future work involves developing an algorithm which selects the lines automatically and evaluates the function.

D] References:

[1] L. O. Chua, "Memristor—The missing circuit element," *IEEE Transactions on Circuit Theory*, vol. 18, no. 5, pp. 507–519, 1971.

[2] D. B. Strukov, G. S. Snider, D. R. Stewart, and R. S. Williams, "The missing memristor found," *Nature*, vol. 453, pp. 80–83, 2008.

[3] J. J. Yang, D. B. Strukov, and D. R. Stewart, "Memristive devices for computing," *Nature Nanotechnology*, vol. 8, no. 1, pp. 13–24, 2013.